CSA0210-C Programming Slot-B

192312552

P Parvash khan

TASK 1: Organise the Program with Header Files and Conditional Compilation

PROGRAM

Create two files: billing.h and main.c.

File 1: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000

long compute_bill(int units);

#endif
```

File 2: main.c

```
#include <stdio.h>
#include "billing.h"

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
```

```
    if (units <= SLAB1_UNITS) {
        paise += units * RATE1_PAISE;
    } else if (units <= 200) {
        paise += SLAB1_UNITS * RATE1_PAISE;
        paise += (units - 100) * RATE2_PAISE;
    } else {
        paise += SLAB1_UNITS * RATE1_PAISE;
        paise += 100 * RATE2_PAISE;
        paise += (units - 200) * RATE3_PAISE;
    }

#if DEBUG
    printf("DEBUG: Units = %d\n", units);
    printf("DEBUG: Bill in paise = %ld\n", paise);
#endif

    return paise;
}

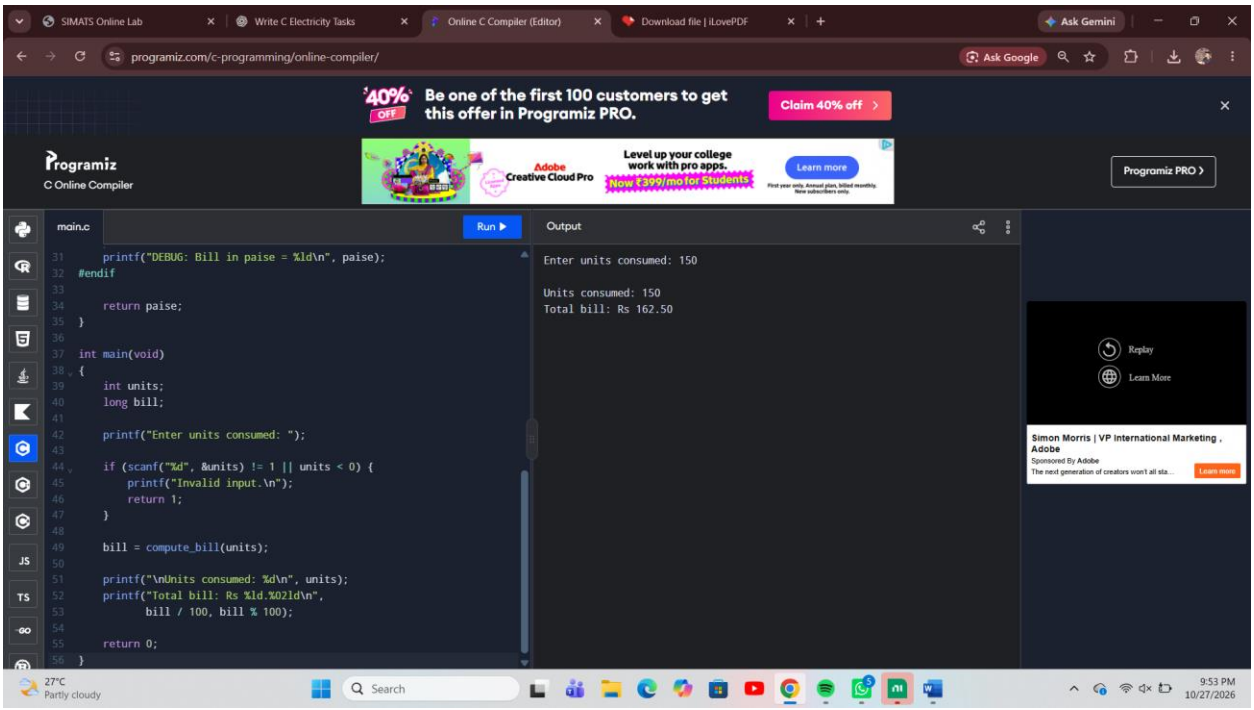
int main(void)
{
    int units = 150;
    long bill = compute_bill(units);

    printf("Units consumed: %d\n", units);
    printf("Total bill: Rs %ld.%02ld\n",
```

```
        bill / 100, bill % 100);


    return 0;
}
```

OUTPUT



EXPLANATION

- #include <stdio.h> includes standard input/output functions.
- billing.h stores constants and the function prototype.
- #ifndef, #define, and #endif prevent repeated inclusion of the header.
- #if DEBUG displays intermediate values only when debugging is enabled.
- gcc -DDEBUG main.c -o billing enables debugging during compilation.

Alternative concept: A runtime condition such as if (debug) can enable or disable debugging while the program runs. Conditional compilation can remove debugging statements from a normal build, avoiding their execution and associated runtime checks.

TASK 2: Read and Print Data Correctly

Question: Design the input and output of an electricity billing program using scanf() and printf(). Use correct format specifiers, validate input, check the return value of scanf(), and print rupees and paise correctly.

ASSUMED INPUT VALUES

Enter household number: 101

Enter category (1-Domestic, 2-Commercial): 1

Enter previous meter reading: 1000

Enter current meter reading: 1150

PROGRAM

```
#include <stdio.h>

int main(void)
{
   int household, category;
   long long previous, current, units;
   long bill, rate, fixed;

   printf("Enter household number: ");
   if (scanf("%d", &household) != 1 || household <= 0) {
      printf("Invalid household number.\n");
      return 1;
   }

   printf("Enter category (1-Domestic, 2-Commercial): ");
   if (scanf("%d", &category) != 1 ||
```

```
    (category != 1 && category != 2)) {
    printf("Invalid category.\n");
    return 1;
}

printf("Enter previous meter reading: ");
if (scanf("%lld", &previous) != 1 || previous < 0) {
    printf("Invalid previous reading.\n");
    return 1;
}

printf("Enter current meter reading: ");
if (scanf("%lld", &current) != 1 || current < previous) {
    printf("Invalid current reading.\n");
    return 1;
}

units = current - previous;

rate = (category == 1) ? 500 : 800;
fixed = (category == 1) ? 5000 : 10000;

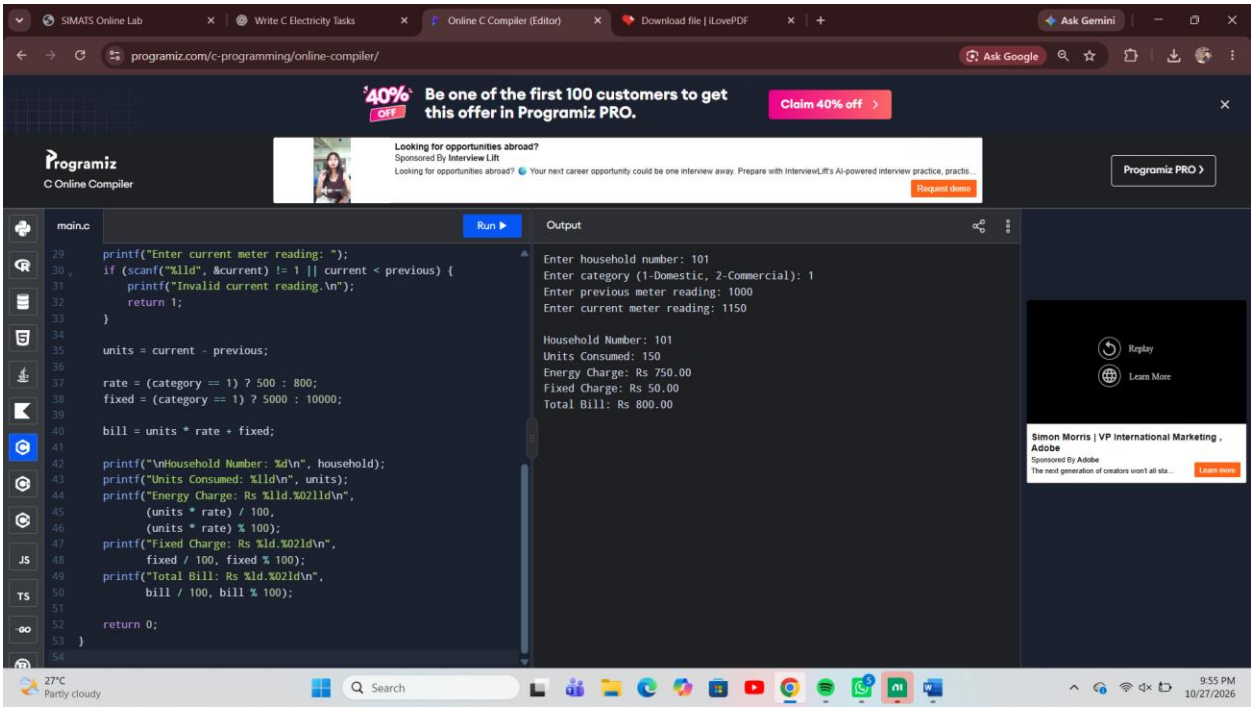
bill = units * rate + fixed;

printf("\nHousehold Number: %d\n", household);
printf("Units Consumed: %lld\n", units);
```

```
    printf("Energy Charge: Rs %lld.%02lld\n",
           (units * rate) / 100,
           (units * rate) % 100);
    printf("Fixed Charge: Rs %ld.%02ld\n",
           fixed / 100, fixed % 100);
    printf("Total Bill: Rs %ld.%02ld\n",
           bill / 100, bill % 100);

    return 0;
}
```

OUTPUT



EXPLANATION

- %d is used for integer values, %lld for long long, and %ld for long.
- scanf() returns the number of successfully read values. Checking it prevents invalid input from being used.

- The program stores monetary values in paise to avoid floating-point rounding errors.
- Domestic rate = 500 paise per unit and fixed charge = 5000 paise. These are illustrative rates for this task.

Alternative concept: Reading an entire line with fgets() and converting it using strtol() allows more detailed validation of extra characters and invalid input. For a line of length \(L\), reading and parsing takes \(O(L)\) time and requires \(O(L)\) buffer space. Direct scanf() is simpler, but checking its return value is essential.

TASK 3: Control Type Conversion and Precision

Question: Analyse implicit conversion, integer division, explicit casting, and rounding errors in electricity billing. Demonstrate at least three expressions that produce unexpected results.

ASSUMED INPUT VALUES

Units consumed: 150

Rate per unit: Rs 2.25

PROGRAM

```
#include <stdio.h>


int main(void)
{
   int units = 150;
   int a = 150, b = 100;
   double rate = 2.25;

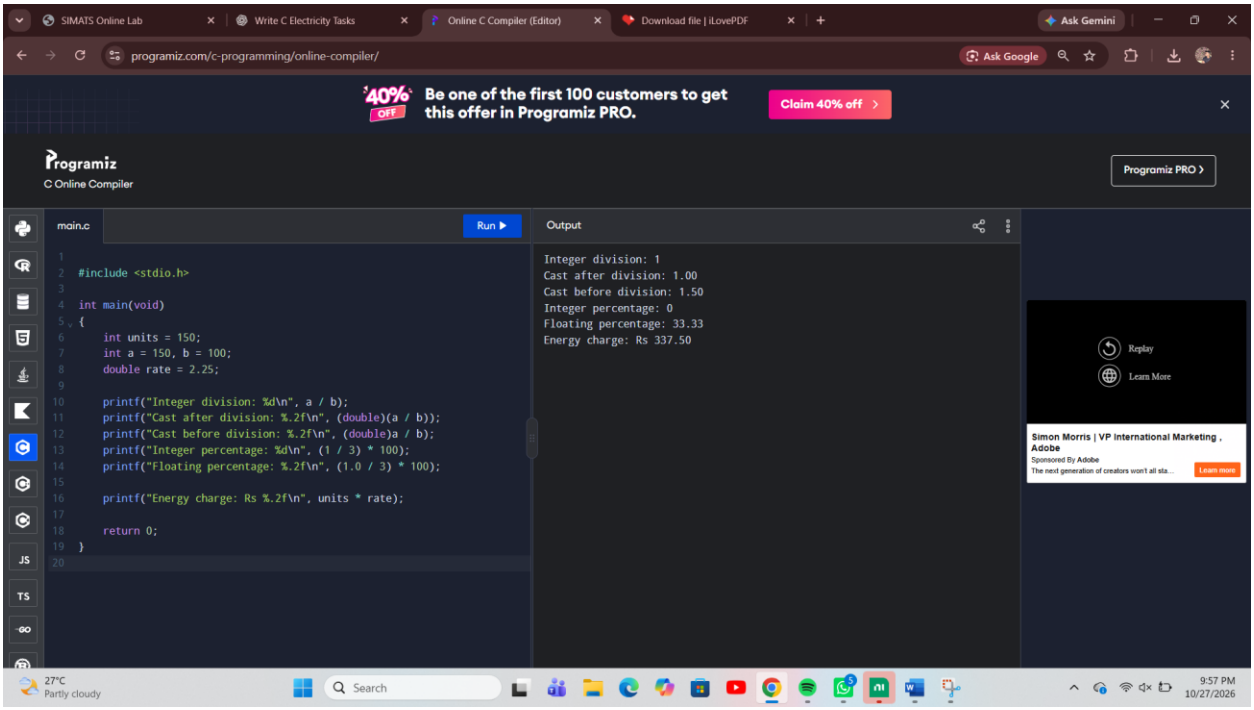

   printf("Integer division: %d\n", a / b);
   printf("Cast after division: %.2f\n", (double)(a / b));
   printf("Cast before division: %.2f\n", (double)a / b);
   printf("Integer percentage: %d\n", (1 / 3) * 100);
```

```
    printf("Floating percentage: %.2f\n", (1.0 / 3) * 100);

    printf("Energy charge: Rs %.2f\n", units * rate);

    return 0;
}
```

OUTPUT



EXPLANATION

1. Integer division: 150 / 100 gives 1, because the fractional part is discarded.
2. Casting after division: (double)(150 / 100) gives 1.00 because the division has already occurred using integers.
3. Casting before division: (double)150 / 100 gives 1.50 because floating-point division is used.
4. Percentage calculation: (1 / 3) * 100 gives 0, while (1.0 / 3) * 100 gives approximately 33.33.

5. Precision: Multiplying integer units by a double rate converts the calculation to floating-point arithmetic, which may introduce a small representation error.

Alternative concept: Using integer paise for all bill calculations and rounding only when converting a fractional charge to paise gives consistent results. Floating-point calculations rounded only at the end may accumulate small errors across thousands of bills. Integer paise uses exact whole-number monetary values, provided the chosen integer type is large enough.

TASK 4: Trace the Compilation Process

Question: Follow a C program through preprocessing, compilation, assembly, and linking. Explain the difference between compile-time, link-time, and run-time errors.

PROGRAM

Save the following code as bill.c.

```
#include <stdio.h>

#define FIXED_CHARGE 50

int main(void)
{
   int units = 150;
   double rate = 2.25;
   double total = FIXED_CHARGE + units * rate;

   printf("Units consumed: %d\n", units);
   printf("Total bill: Rs %.2f\n", total);

   return 0;
}
```

COMPILATION COMMANDS

Run these commands in the terminal in the given order.

1. Preprocessing

gcc -E bill.c -o bill.i

Output: bill.i contains expanded header files and macros.

2. Compilation to assembly

gcc -S bill.i -o bill.s

Output: bill.s contains assembly instructions.

3. Assembly to object file

gcc -c bill.s -o bill.o

Output: bill.o contains machine code and object-file information.

4. Linking

gcc bill.o -o bill

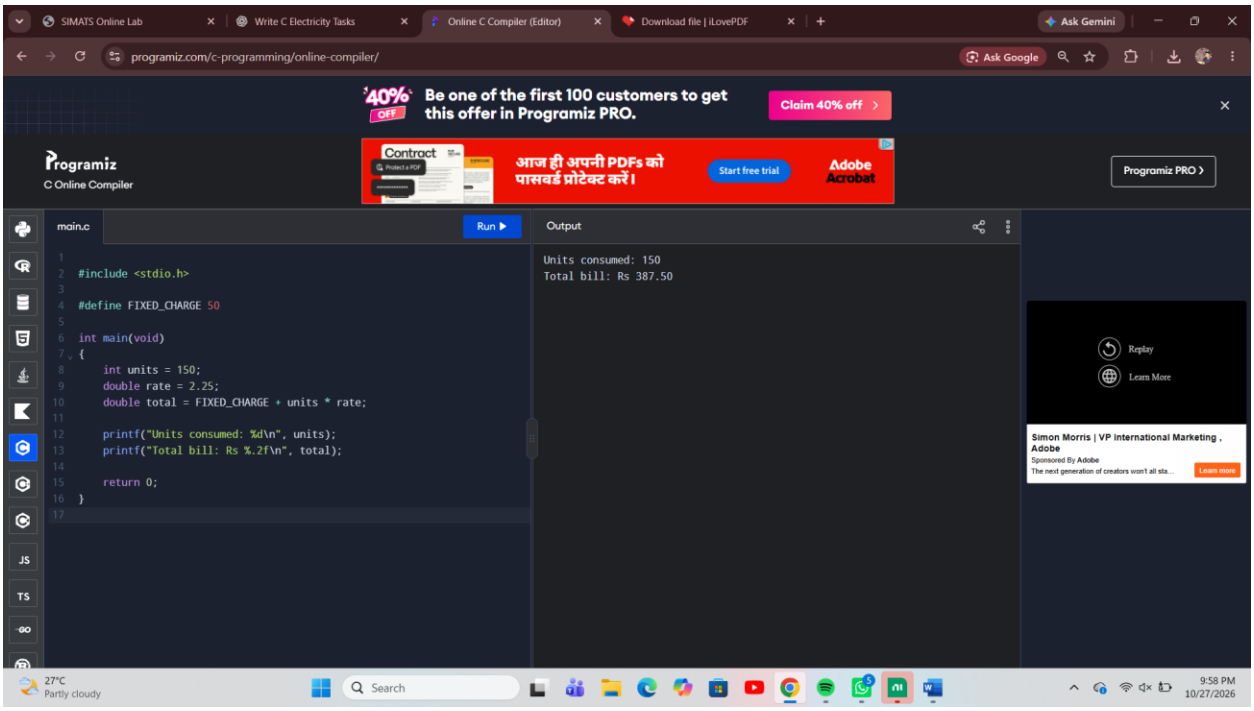
Output: Creates the executable program named bill.

5. Execute the program

./bill

On Windows, use bill.exe or run bill in the appropriate terminal.

OUTPUT



EXPLANATION

- Preprocessing: Expands #include files and macros such as FIXED_CHARGE.
- Compilation: Translates C code into assembly and checks for syntax and type errors.
- Assembly: Converts assembly instructions into an object file.
- Linking: Combines object files and required libraries to create the executable.